LESSON 04 · PYTEST · 50 MIN

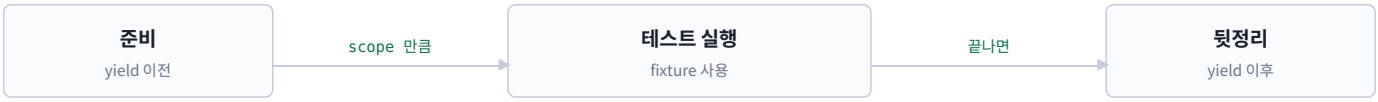
4교시 pytest 활용 — fixture · parametrize · mock

fixture의 scope를 넓히면 빨라지지만 테스트끼리 같은 것을 나눠 쓰게 됩니다. 속도와 격리를 맞바꾸는 감각을 잡고, 케이스를 늘리는 법과 외부 의존을 끊는 법을 익힙니다.

개념 12분 · 실습 28분 · 정리와 질문 10분

OVERVIEW

fixture 한 번의 수명



scope 가 **function** 이면 이 한 바퀴가 테스트마다 돕니다. **module** 이면 파일마다, **session** 이면 전체에서 한 번입니다. 횟수가 줄수록 빨라지고, 대신 테스트끼리 같은 객체를 나눠 씁니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`@pytest.fixture`	테스트가 인자로 받아 쓰는 준비물	중복되는 준비 코드를 뺄 때
`yield`	앞은 준비, 뒤는 뒷정리	파일·연결처럼 닫아야 하는 것
`scope="module" / "session"`	준비를 몇 번 할지	느린 준비물(DB·서버)일 때
`conftest.py`	여러 파일이 공유하는 fixture 보관소	import 없이 자동으로 찾아 준다
`--setup-show`	fixture 준비·정리 순서를 눈으로 확인	scope 를 이해할 때 최고의 명령
`@pytest.mark.parametrize`	입력 목록으로 테스트를 늘린다	같은 로직, 다른 값
`tmp_path / monkeypatch`	임시 폴더 / 값을 임시로 바꾸기	파일·환경변수 다룰 때
`caplog`	기록된 로그를 검증한다	1·2교시에서 심은 로그를 확인
`mocker.patch("경로")`	외부 호출을 가짜로 대체	네트워크·결제·시간

FILES

실습용 파일

```
conftest.py
import pytest
from cart import Cart


@pytest.fixture
def cart():
    print("\n  [준비] 새 장바구니")
    c = Cart()
    yield c
    print("  [정리] 장바구니 비움")
    c.items.clear()
```

HANDS-ON

실습 순서

STEP 01 중복 준비 코드를 fixture 로 빼기

테스트마다 반복되던 두 줄을 없앱니다.

```
# 위 '실습용 파일'의 conftest.py 를 tests/ 폴더에 만듭니다
# test_cart.py 를 아래처럼 바꿉니다
def test_total_sums_items(cart):
    cart.add("book", 12000, 2)
    assert cart.total() == 24000

pytest -v -s
```

확인 포인트

- [ ] Cart() 를 직접 만드는 줄이 사라졌다
- [ ] -s 옵션 덕분에 [준비] / [정리] 출력이 보인다
- [ ] 테스트 개수만큼 준비와 정리가 반복된다

conftest.py 는 import 하지 않습니다. pytest가 같은 폴더와 상위 폴더에서 자동으로 찾습니다. 이름을 정확히 지켜야 하고, 위치가 곧 적용 범위입니다.

STEP 02 scope 를 눈으로 확인하기 — 오늘의 핵심 (1)

준비가 몇 번 일어나는지 직접 셉니다.

```
# conftest.py 의 fixture 에 scope 를 붙여 가며 실행
@pytest.fixture(scope="function")   # 기본값
pytest --setup-show

@pytest.fixture(scope="module")
pytest --setup-show

@pytest.fixture(scope="session")
pytest --setup-show
```

확인 포인트

- [ ] function: SETUP 이 테스트마다 나온다
- [ ] module: 파일마다 한 번으로 줄어든다
- [ ] session: 전체에서 한 번만 나온다
- [ ] 실행 시간이 조금씩 줄어드는 것도 확인

> **--setup-show** 는 fixture 를 이해하는 가장 빠른 길입니다. 어떤 fixture 가 언제 준비되고 언제 정리되는지 실행 순서 그대로 보여 줍니다.

STEP 03 넓은 scope 의 대가 확인하기 — 오늘의 핵심 (2)

공유하면 무엇이 위험해지는지 재현합니다.

```
# scope="session" 으로 둔 상태에서 테스트를 두 개 만듭니다
def test_first(cart):
    cart.add("book", 1000)
    assert len(cart.items) == 1

def test_second(cart):
    assert len(cart.items) == 0    # 새 장바구니를 기대

pytest -v

# 이제 scope 를 function 으로 되돌리고 다시 실행
pytest -v
```

확인 포인트

- [ ] session: test_second 가 실패한다 — 앞 테스트가 담은 물건이 남아 있다
- [ ] function: 둘 다 통과한다
- [ ] test_second 만 따로 돌리면 session 에서도 통과한다

> **따로 돌리면 통과하는데 같이 돌리면 실패하는 테스트**는 원인 찾기가 가장 어렵습니다. 그래서 기본은 function 이고, 준비가 정말 느린 것(DB 컨테이너·서버 기동)에만 넓은 scope 를 씁니다.

STEP 04 parametrize 로 케이스 늘리기

같은 로직에 여러 값을 넣습니다.

```
@pytest.mark.parametrize("price, qty, expected", [
    (12000, 2, 24000),
    (0, 5, 0),
    (1000, 1, 1000),
])
def test_total(cart, price, qty, expected):
    cart.add("item", price, qty)
    assert cart.total() == expected

pytest -v
```

확인 포인트

- [ ] 함수는 하나인데 테스트가 3개로 표시된다
- [ ] 이름이 test_total[12000-2-24000] 처럼 값과 함께 나온다
- [ ] 하나가 실패해도 어떤 조합인지 대괄호로 바로 알 수 있다

> 데코레이터를 **두 개 겹치면 곱셈**이 됩니다. 3개 × 2개면 6개가 만들어지니, 무심코 늘리면 실행 시간이 급격히 불어납니다.

STEP 05 tmp_path 와 monkeypatch

파일과 환경변수를 안전하게 다룹니다.

```
def test_save_report(tmp_path):
    f = tmp_path / "report.txt"
    f.write_text("매출 1000", encoding="utf-8")
    assert f.read_text(encoding="utf-8").startswith("매출")

def test_env(monkeypatch):
    monkeypatch.setenv("ENV", "test")
    import os
    assert os.getenv("ENV") == "test"

pytest -v
```

확인 포인트

- [ ] tmp_path 는 테스트마다 새 임시 폴더를 준다
- [ ] 테스트가 끝나면 원래 환경변수로 자동 복구된다

> **실제 프로젝트 폴더에 파일을 쓰는 테스트는 만들지 마세요.** 뒷정리를 잊으면 다음 실행이 오염됩니다. tmp_path 는 이 문제를 통째로 없애 줍니다.

STEP 06 caplog 으로 로그 검증하기

1·2교시에 심은 로그가 실제로 남는지 테스트합니다.

```
import logging

def test_error_is_logged(cart, caplog):
    with caplog.at_level(logging.ERROR):
        try:
            cart.total()
        except Exception:
            logging.getLogger("app").exception("빈 장바구니 결제 시도")

    assert "빈 장바구니" in caplog.text
    assert caplog.records[0].levelname == "ERROR"

pytest -v
```

확인 포인트

- [ ] caplog.text 에 메시지가 들어 있다
- [ ] 레벨까지 확인할 수 있다

> 로깅과 테스트가 따로 노는 주제가 아니라는 지점입니다. **"실패하면 ERROR 로그가 남는다"** 도 지켜야 할 동작이라면 테스트로 고정해 두는 것이 맞습니다.

STEP 07 mock 으로 외부 의존 끊기

느리고 불안정한 외부 호출을 대체합니다.

```
pip install pytest-mock

# payment.py
import requests
def charge(amount):
    r = requests.post("https://api.example.com/pay", json={"amount": amount})
    return r.json()

# test_payment.py
def test_charge_called_with_amount(mocker):
    fake = mocker.patch("payment.requests.post")
    fake.return_value.json.return_value = {"status": "ok"}

    from payment import charge
    assert charge(12000) == {"status": "ok"}
    assert fake.call_count == 1
    assert fake.call_args.kwargs["json"]["amount"] == 12000

pytest -v
```

확인 포인트

- [ ] 네트워크 없이도 통과한다
- [ ] 몇 번 불렀는지, 무엇을 넘겼는지 검증할 수 있다

> **patch 경로는 정의된 곳이 아니라 쓰이는 곳입니다.** requests.post 가 아니라 **payment.requests.post** 를 패치한 이유가 이것이고, mock 실습에서 가장 많이 틀리는 지점입니다.

STEP 08 커버리지 측정

테스트가 닿지 않은 곳을 찾습니다.

```
pip install pytest-cov
pytest --cov=src --cov-report=term-missing
pytest --cov=src --cov-report=html
# htmlcov/index.html 을 브라우저로 열어 봅니다
```

확인 포인트

- [ ] Missing 칸에 실행되지 않은 줄 번호가 나온다
- [ ] HTML 리포트에서 빨간 줄이 안 돌아간 코드다

> **커버리지는 목표가 아니라 지도입니다.** 숫자를 채우려고 의미 없는 테스트를 늘리면 "통과하는데 아무것도 보장하지 않는 테스트"만 쌓입니다. Missing 줄을 보고 **정말 위험한 곳부터** 채우세요.

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
`fixture 'cart' not found`	conftest.py 이름·위치가 틀림	파일명 정확히 conftest.py · 테스트와 같은 폴더나 상위 폴더에
따로 돌리면 통과, 같이 돌리면 실패	넓은 scope 로 상태를 공유	scope 를 function 으로 되돌리거나 fixture 에서 상태를 초기화
`mock` 을 걸었는데 진짜가 호출됨	patch 경로가 정의처 기준	쓰이는 모듈 기준으로 경로 지정 (예: payment.requests.post)
`parametrize` 인자 이름 불일치	문자열과 함수 인자가 다름	"a, b" 와 def test(a, b) 의 이름을 정확히 맞춤
커버리지가 `0%` 로 나옴	--cov 경로가 실제 소스와 다름	--cov=src 처럼 소스 폴더를 정확히 지정

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

- [ ] fixture 의 기본 scope 는 무엇이고 왜 그것이 기본일까요?
- [ ] --setup-show 는 무엇을 보여 주나요?
- [ ] 따로 돌리면 통과하는데 같이 돌리면 실패한다면 무엇을 의심해야 하나요?
- [ ] parametrize 데코레이터를 두 개 겹치면 테스트는 몇 개가 되나요?
- [ ] mock 의 patch 경로는 무엇을 기준으로 적나요?

한 줄 정리

scope 는 속도와 격리를 맞바꾸는 손잡이입니다. 기본은 좁게, 정말 느린 것만 넓게.

다음 시간 → 5교시 — 이 테스트를 내 컴퓨터 밖에서 자동으로 돌리기

WORKSHEET

막힌 STEP 번호 · 화면에 나온 오류 메시지 · 물어볼 것